UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Aprendizado Não-Supervisionado

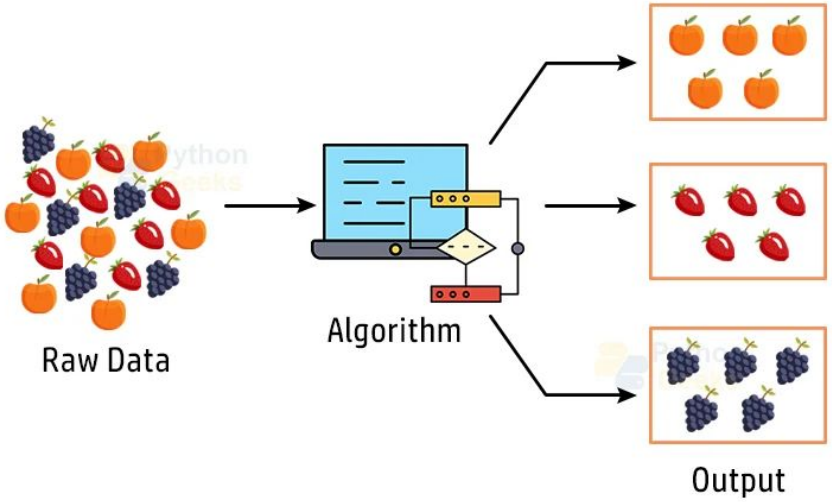
K-Means

Prof. Dr. Silvan Ferreira

METRÓPOLE DIGITAL

Motivação

- Você tem milhares de artigos científicos em texto: deseja agrupar em tópicos
- Você trabalha em marketing: quer segmentar clientes em grupos com padrões de compra semelhantes
- Você estuda comportamento animal: deseja agrupar automaticamente diferentes tipos de movimentos observados em vídeos
- Tema comum: **dados sem rótulos** podem ser agrupados em clusters, revelando **estruturas ocultas (latentes)**

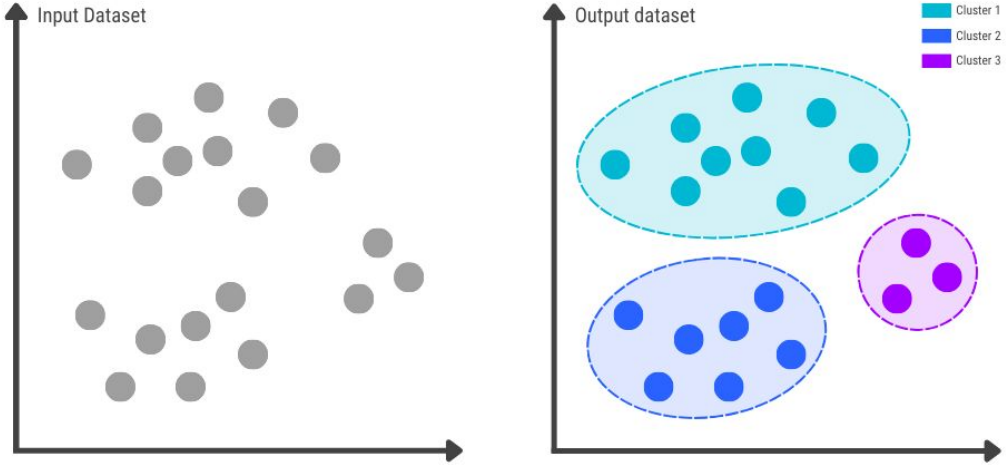


Clustering

- Assumimos que temos dados $\mathcal{D} = \{x^{(1)}, x^{(2)}, \ldots, x^{(N)}\}, \quad x^{(n)} \in \mathbb{R}^d$
- Cada ponto vive em um espaço Euclidiano: $x^{(n)} \in \mathbb{R}^d$
- Supomos que existem K classes (padrões) nos dados
- Hipótese: pontos da mesma classe são próximos em distância Euclidiana
- Pergunta: como identificar essas classes sem conhecer os rótulos?

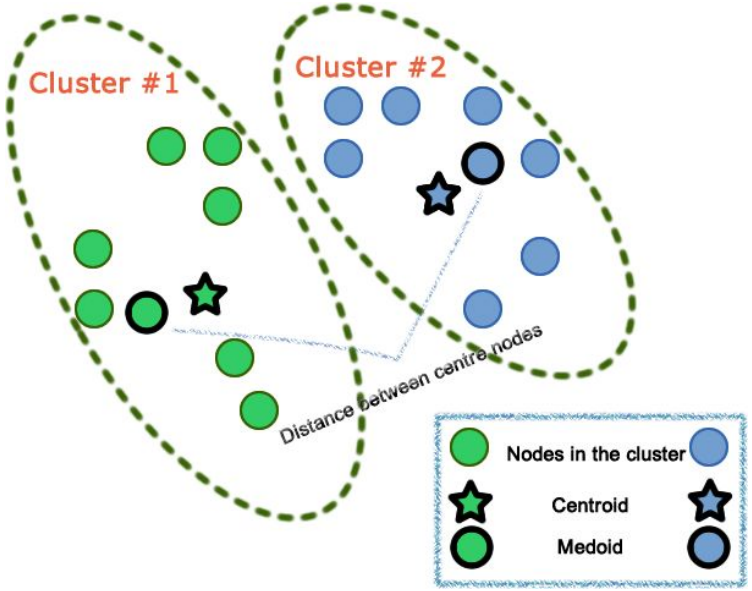
$$\text{Objetivo: encontrar } \{C_1, \ldots, C_K\} \quad \text{tais que} \quad x^{(i)}, x^{(j)} \in C_k \implies \|x^{(i)} - x^{(j)}\| \text{ é pequeno}$$

Clustering



K-Means

- **Suposição:** existem k clusters
- Cada ponto deve estar próximo ao centróide
- Se soubéssemos as atribuições, calcular médias seria fácil
- Se soubéssemos as médias, atribuir pontos seria fácil
- **Heurística prática:** inicializar aleatoriamente e alternar atribuição ↔ atualização



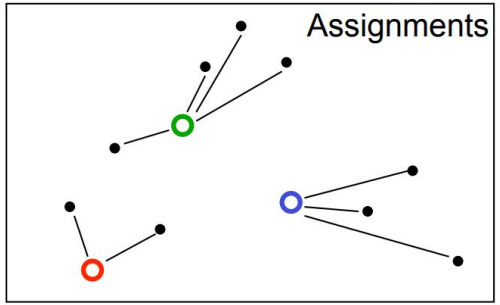
K-Means

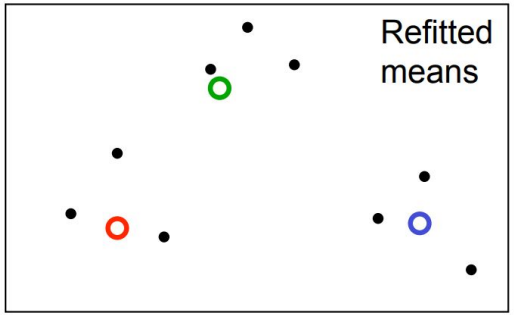
- **Inicialização:** escolher aleatoriamente k centróides
- O algoritmo iterativamente alterna entre dois passos:
  - **Passo 1 (Assignment):** atribuir cada ponto ao centróide mais próximo

$$x^{(n)} \in C_k \quad \Longleftrightarrow \quad k = \arg\min_j \|x^{(n)} - \mu_j\|^2 \qquad r_k^{(n)} = \begin{cases} 1 & \text{se } k = \arg\min_j \|x^{(n)} - \mu_j\|^2 \\ 0 & \text{caso contrário} \end{cases}$$

  - **Passo 2 (Refitting):** atualizar cada centróide para a média dos pontos atribuídos

$$\mu_k = \frac{1}{|C_k|} \sum_{x \in C_k} x$$





K-Means

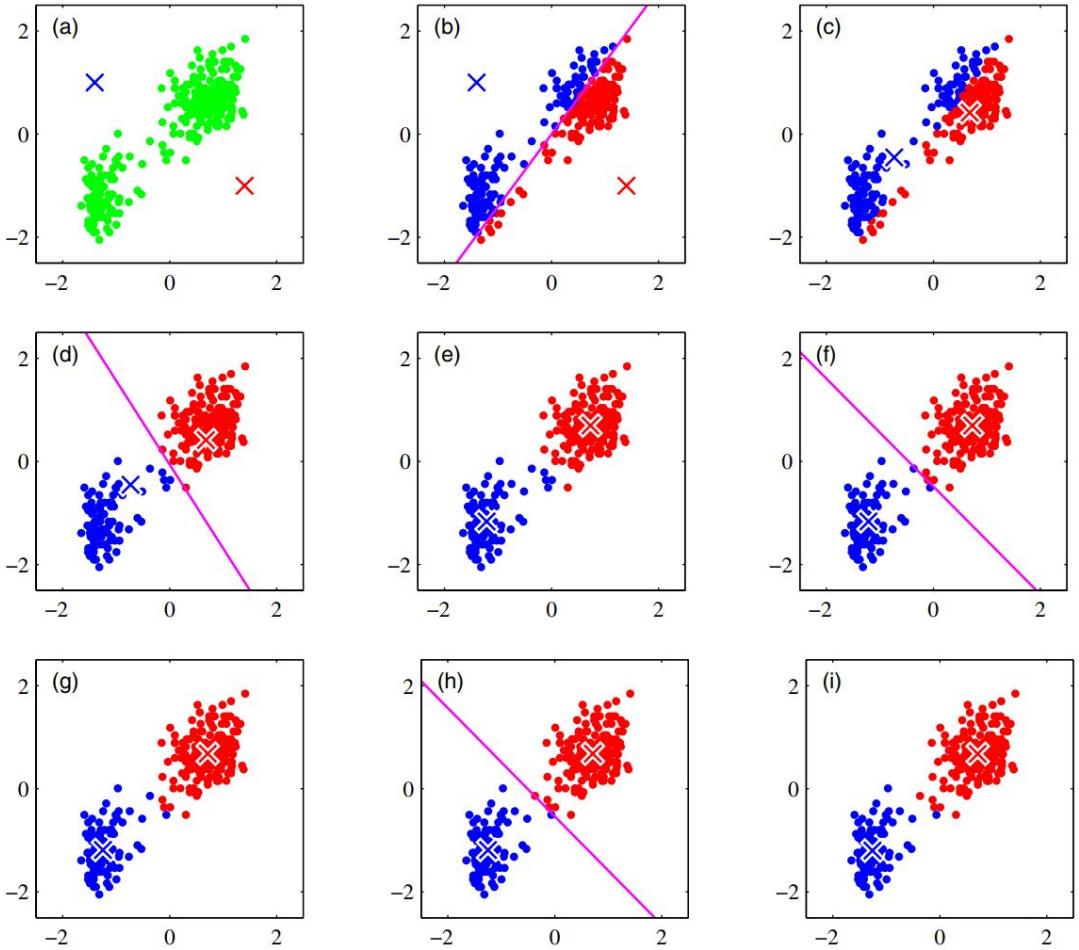
O que o K-Means realmente otimiza?

- **Objetivo:** encontrar centróides {m} e atribuições {r}
- Minimizar a soma das distâncias quadráticas entre pontos e seus centróides
- Otimização é feita via “coordinate descent”
  - Fixar centróides → otimizar atribuições
  - Fixar atribuições → otimizar centróides

$$\sum_{k=1}^{K} r_k^{(n)} = 1 \quad \forall n, \qquad r_k^{(n)} \in \{0, 1\}$$

$$J(\{\mu\}, \{r\}) = \sum_{n=1}^{N} \sum_{k=1}^{K} r_k^{(n)} \, \|x^{(n)} - \mu_k\|^2$$

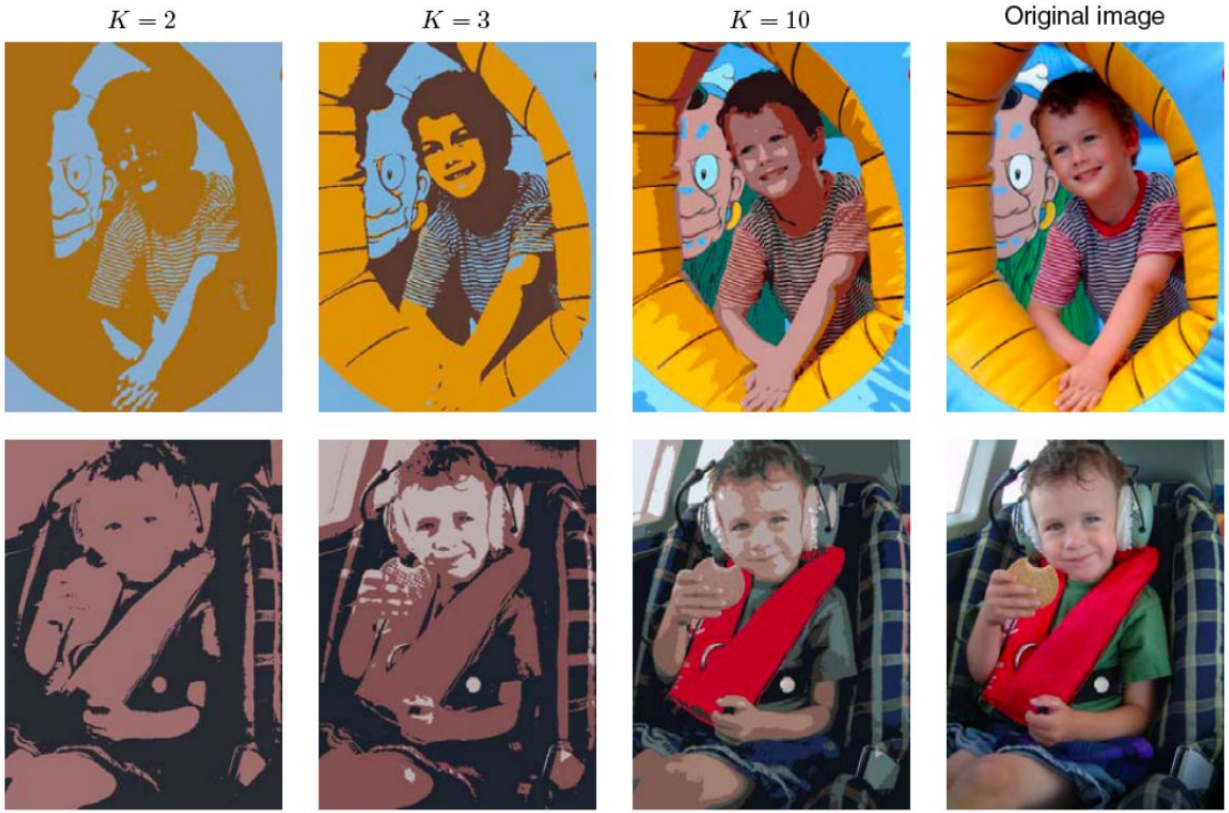
K-Means



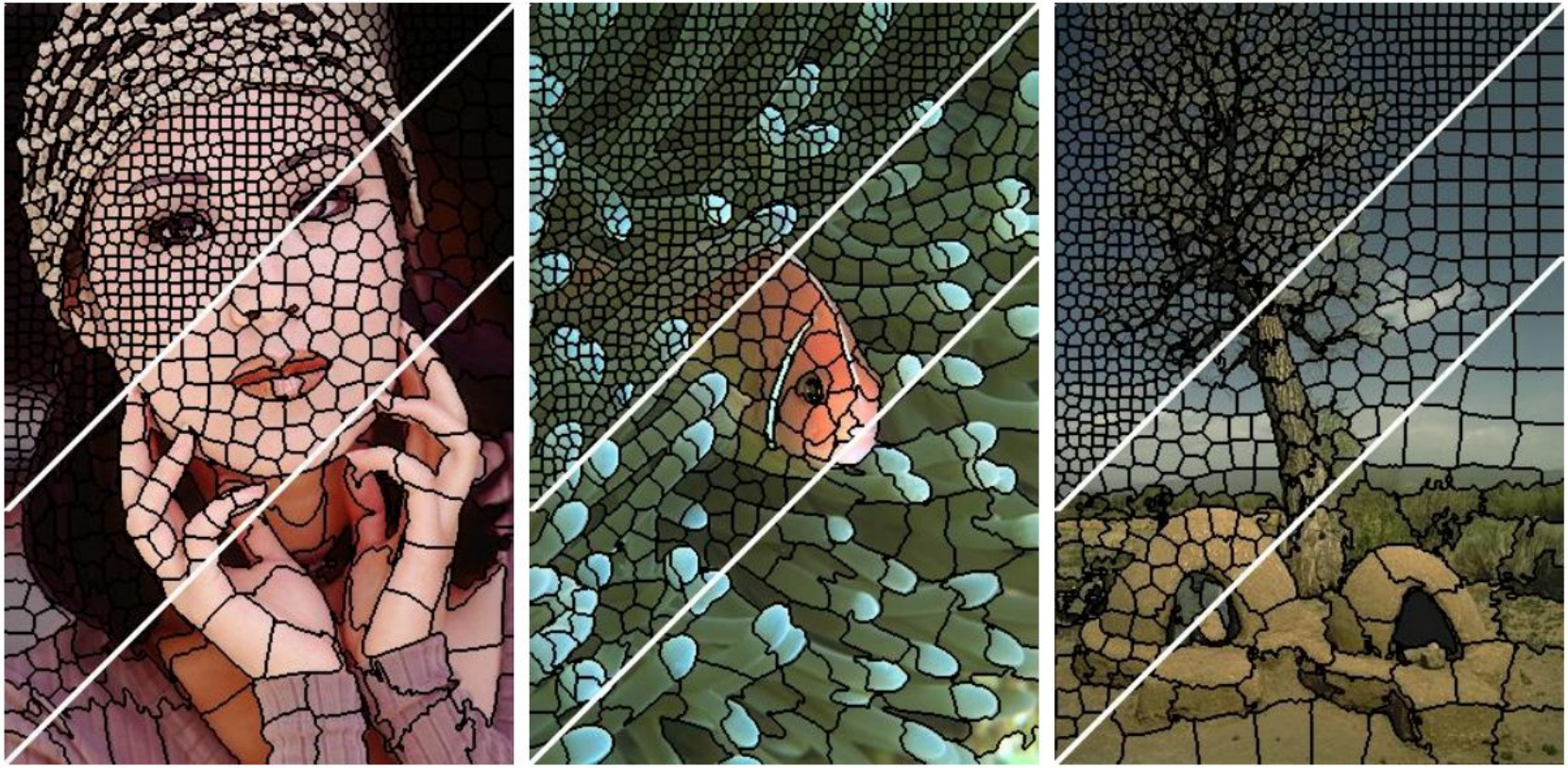
Links Úteis

- https://www.naftaliharris.com/blog/visualizing-k-means-clustering/

K-Means para Quantização Vetorial



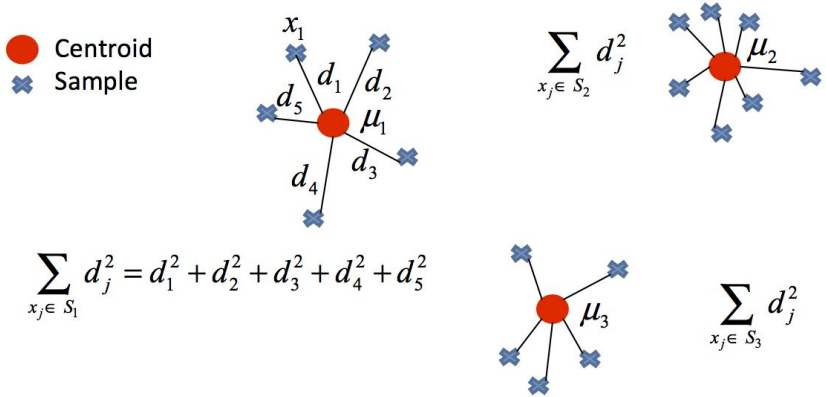
K-Means para Segmentação



Convergência

- Cada vez que uma atribuição é alterada, a soma das distâncias quadráticas (J) diminui
- Cada vez que um centróide é movido, J também diminui
- Como J é limitado inferiormente (≥ 0), o algoritmo deve convergir
- Garantia: convergência para um mínimo local (não necessariamente o global)

$$J(\{\mu\},\{r\}) = \sum_{n=1}^{N}\sum_{k=1}^{K} r_k^{(n)} \|x^{(n)} - \mu_k\|^2$$

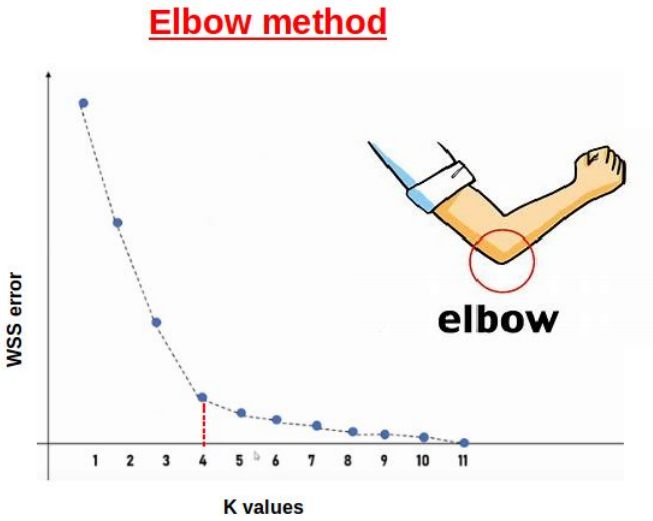


$$\min_S E(\mu_i) = \sum_{x_j \in S_1} d_j^2 + \sum_{x_j \in S_2} d_j^2 + \sum_{x_j \in S_3} d_j^2$$

Método do Cotovelo

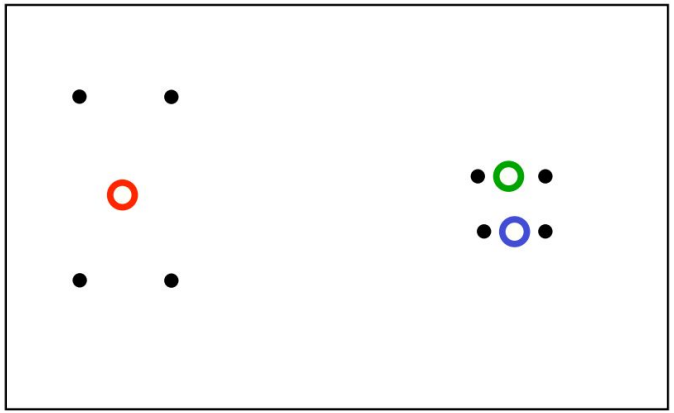
- **Questão:** como escolher o número de clusters k?
- **Ideia:** calcular a função objetivo (ou inércia) para diferentes valores de k
- **Inércia:** soma das distâncias quadráticas de cada ponto ao seu centróide, também chamada de **Within-Cluster Sum of Squares (WSS)**
- Procuramos o ponto onde a redução deixa de ser significativa
- Esse ponto de inflexão é chamado de “cotovelo”

$$\text{Inércia}(k) = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$



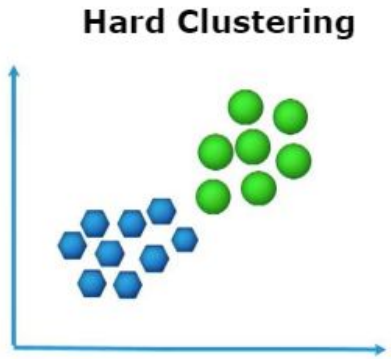
Mínimos Locais

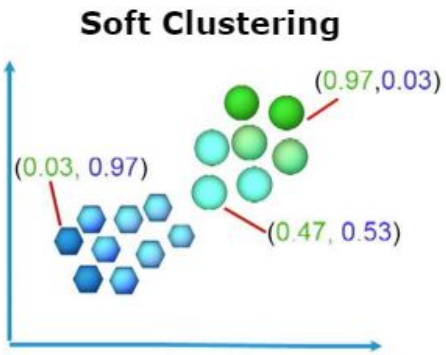
- A função objetivo J é não convexa
- Coordinate descent não garante convergência ao mínimo global
- O algoritmo pode ficar preso em mínimos locais
- Estratégias para lidar com isso:
    - Rodar o K-Means várias vezes com inicializações diferentes



Soft K-Means

- No K-Means clássico (hard), cada ponto pertence a exatamente um cluster
- No Soft K-Means, cada ponto tem **probabilidades de pertencimento** a cada cluster
- Isso permite modelar incerteza e sobreposição entre clusters
- Iterativo como o K-Means, mas usa atribuições suaves (responsabilidades)





Soft K-Means

- **Inicialização:** escolher aleatoriamente k centróides
- O algoritmo iterativamente alterna entre dois passos:
  - **Passo 1 (Assignment):** calcular responsabilidades

$$r_k^{(n)} = \frac{\exp\left(-\beta \|x^{(n)} - \mu_k\|^2\right)}{\sum_{j=1}^{K} \exp\left(-\beta \|x^{(n)} - \mu_j\|^2\right)}$$

  - **Passo 2 (Update):** atualizar centróides com média ponderada pelas atribuições

$$\mu_k = \frac{\sum_{n=1}^{N} r_k^{(n)} x^{(n)}}{\sum_{n=1}^{N} r_k^{(n)}}$$

  - Repetir até convergência

Notebook

- https://colab.research.google.com
- https://github.com/silvaan/unsupervised-learning
- 02 - K-Means.ipynb